



Pontificia Universidad Católica del Ecuador

Ingeniería de Software

Documentación



Autor: Joshoa Castañeda

Docente: Ing. Jean Carlos Velesaca Macias.

Asignatura:

PRACTICAS PRE-PROFESIONALES

Manabí – Ecuador

Julio 2025



Tabla

BACK-END	4
Base de Datos.....	4
❖ Creación de Tablas	4
❖ Procedimientos Almacenados	6
❖ Relaciones Clave (Claves Foráneas).....	9
Api	10
❖ Archivos de Configuración.....	10
❖ Endpoints	11
FRONT-END.....	16
➤ Estructura de Componentes.....	16
➤ Descripción del Componente SedeComponent	20
➤ Descripción del Archivo de rutas (AppRoutingModule).....	23
COMO INICIAR EL PROYECTO	25
➤ Pre-requisitos.....	25
➤ Iniciar el Back-end.....	25



➤	Iniciar el Front-end	25
---	-----------------------------------	----





BACK-END

Base de Datos

❖ Creación de Tablas

La base de datos (plantilla_automatizada) está diseñada en MySQL, sirve para gestionar información relacionada con sedes, áreas funcionales, carpetas documentales y formatos de archivo digital o físico. Incluye relaciones jerárquicas y permite registrar, consultar, actualizar y eliminar registros mediante procedimientos almacenados.

Consta de las siguientes tablas:

➤ Sede

Registra información en los siguientes campos

- **id_sede:** Identificador único de la sede.
- **nombre_sede:** Nombre de la sede.
- **ciudad:** Ciudad donde se ubica.

➤ Áreas

Registra información en los siguientes campos

- **area_id:** Identificador del área.



- **nombre:** Nombre del área.
- **id_sub_area:** Referencia a otra área que actúa como subárea.
- **departamento:** Departamento al que pertenece.
- **id_sede:** Llave foránea hacia la tabla Sede.

➤ Carpeta

Registra información en los siguientes campos

- **id_carpeta:** Identificador único.
- **nombre:** Nombre de la carpeta.
- **id_sede:** Llave foránea hacia Sede.
- **id_sub_carpeta:** Subcarpeta (auto-relación).

➤ Formato

Registra información en los siguientes campos

- **formato_id:** Identificador único.
- **activo_de_informacion:** Descripción del contenido.
- **id_area:** Área a la que pertenece.
- **periodo_de_conservacion_anio:** Años de conservación.



- **ubicacion_descripcion:** Ubicación física o digital.
- **tipo_documento:** Tipo de archivo (PDF, DOCX, etc.).
- **tipo_grupo:** Clasificación funcional (Legal, Técnico, etc.).
- **tamano:** Tamaño del archivo.
- **nombre_archivo:** Nombre del archivo.
- **fecha_subida:** Fecha de carga.
- **tipo_etiqueta:** Etiqueta organizativa.
- **id_carpeta:** Carpeta donde se encuentra.
- **tipo_activo:** Tipo de activo documental.
- **confidencialidad:** Valor booleano sobre privacidad.
- **criticidad:** Nivel de importancia (Alta, Media, Baja).

❖ Inserción de datos de Prueba

Se insertan registros de ejemplo en todas las tablas para facilitar las pruebas y la validación del modelo de datos.

❖ Procedimientos Almacenados

Organizados por operaciones CRUD (Create, Read, Update, Delete).

➤ Consultar – Listar Datos



- **listar_sedes()**

Lista todas las sedes.

- **consultar_sedes_por_nombre(nombre)**

Filtra sede por su nombre exacto.

- **listar_areas()**

Lista todas las áreas.

- **consultar_area_por_id(id)**

Retorna los datos de un área según su ID.

- **listar_carpetas()**

Lista todas las carpetas.

- **consultar_carpeta_por_id(id)**

Consulta una carpeta específica por ID.

- **listar_formatos()**

Muestra todos los formatos documentales.

- **consultar_formato_por_id(id)**

Devuelve un formato por su identificador.

➤ **Insertar Datos**



- **insertar_sede(nombre, ciudad)**

Inserta una nueva sede.

- **insertar_area(nombre, sub_area, departamento, sede)**

Inserta un nuevo registro en Areas.

- **insertar_carpeta(nombre, sede, sub_carpeta)**

Crea una nueva carpeta.

- **insertar_formato(todos los parámetros de la tabla)**

Inserta un nuevo formato documental con todos sus metadatos.

➤ **Actualizar Datos**

- **actualizar_sede(id, nombre, ciudad)**

Modifica una sede existente.

- **actualizar_area(id, nombre, sub_area, departamento, sede)**

Actualiza los datos de un área.

- **actualizar_carpeta(id, nombre, sede, sub_carpeta)**

Cambia la información de una carpeta.

- **actualizar_formato(todos los parámetros de la tabla)**

Actualiza un registro existente de Formato.



➤ **Eliminar Datos**

- **eliminar_sede(id)**

Elimina una sede por ID.

- **eliminar_area(id)**

Borra un área específica.

- **eliminar_carpeta(id)**

Elimina una carpeta existente.

- **eliminar_formato(id)**

Elimina un documento del sistema.

❖ **Relaciones Clave (Claves Foráneas)**

- **Areas.id_sub_area → Areas.area_id:** Relación jerárquica entre áreas.
- **Areas.id_sede → Sede.id_sede:** Un área pertenece a una sede.
- **Carpeta.id_sede → Sede.id_sede:** Una carpeta pertenece a una sede.
- **Carpeta.id_sub_carpeta → Carpeta.id_carpeta:** Jerarquía de carpetas.
- **Formato.id_area → Areas.area_id:** Un documento está asociado a un área.



- **Formato.id_carpeta → Carpeta.id_carpeta:** Un documento se guarda en una carpeta.

Api

permite realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre las entidades de la base de datos (plantilla_automatizada) usando procedimientos almacenados y respuestas en formato JSON. Está construida en PHP y diseñada para comunicarse con un frontend (en Angular) a través de peticiones HTTP.

❖ Archivos de Configuración

- **.htaccess**

Función: Habilita la reescritura de URLs amigables, permitiendo acceder a listar en lugar de listar.php.

- **cors.php**

Función: Habilita CORS (Cross-Origin Resource Sharing) para permitir que aplicaciones frontend accedan a la API sin restricciones de origen.

- **db.php**

Función: Establece la conexión con la base de datos utilizando PDO con manejo de excepciones.



❖ Endpoints

➤ Areas

- **listar**

Método: GET

Ruta: API/Area/listar

Descripción: Retorna todas las áreas de la base de datos.

- **consultar**

Método: GET

Ruta: API/Area/consultar?area_id= (ejemplo 1, 2, 3)

Descripción: Consulta una única área según su area_id.

Parámetro obligatorio: area_id

- **crear**

Método: POST

Ruta: API/Area/crear

Agregar: nombre, departamento, id_sede, id_sub_area (opcional)



- **actualizar**

Método: PUT

Ruta: API/Area/actualizar

Campos obligatorios: area_id, nombre, departamento, id_sede

Campo opcional: id_sub_area (porque es null predeterminadamente)

- **eliminar**

Método: DELETE

Ruta: API/Area/eliminar

Campo obligatorio: area_id

➤ **Carpeta**

- **listar**

Método: GET

Ruta: API/Carpeta/listar

- **consultar**

Método: GET

Ruta: API/Carpeta/consultar?id_carpeta= (ejemplo 1, 2, 3)



Parámetro obligatorio: id_carpeta

- **crear**

Método: POST

Ruta: API/Carpeta/crear

Campos obligatorios: nombre, id_sede

Campo opcional: id_sub_carpeta

- **actualizar**

Método: PUT

Ruta: API/Carpeta/actualizar

Campos obligatorios: id_carpeta, nombre, id_sede

Campo opcional: id_sub_carpeta

- **eliminar**

Método: DELETE

Ruta: API/Carpeta/eliminar

Campo obligatorio: id_carpeta



➤ **Formato**

- **listar**

Método: GET

Ruta: API/Formato/listar

- **consultar**

Método: GET

Ruta: API/Formato/consultar?formato_id= (ejemplo 1, 2, 3)

Parámetro obligatorio: formato_id

- **crear**

Método: POST

Ruta: API/Formato/crear

- **actualizar**

Método: PUT

Ruta: API /Formato/actualizar



- **eliminar**

Método: DELETE

Ruta: API /Formato/eliminar

➤ **Sedes**

- **Listar**

Método: Get

Ruta: API/sede/listar.php

- **Consultar**

Método: Get

Ruta: API/sede/consultar_por_nombre.php?nombre_sede= (Ejemplo Sede Norte)

Parámetro obligatorio: nombre_sede

- **Crear**

Método: POST

Ruta: API/sede/crear.php

Verifica que se envíen nombre_sede y ciudad.

- **Actualizar**



Método: PUT

Ruta: API/sede/actualizar.php

Verifica que se envíen id_sede, nombre_sede y ciudad.

- **Eliminar**

Método: DELETE

Ruta: API/sede/eliminar.php

Campo obligatorio: id_sede

FRONT-END

Está desarrollado en Angular (v 11.0.7) y ngx-bootstrap (v 6.2.0). Consta de un panel de navegación donde se encuentran cuatro opciones: Sede, Área, Carpeta y Formato. Todas presentan la misma estructura en forma de matriz para agregar y visualizar información.

➤ **Estructura de Componentes**

El proyecto consta de los siguientes 5 componentes funcionales y 1 pipe personalizado:

1. PanelNavegacionComponent

- **Ubicación:** src/app/panel-navegacion/panel-navegacion.component.ts



- **Función:**

Barra de navegación que permite al usuario cambiar entre las secciones: Sede, Área, Carpeta y Formato.

2. SedeComponent

- **Ubicación:** src/app/sede/sede.component.ts

- **Función:**

Permite agregar, listar, buscar y eliminar sedes.

Utiliza el FilterSedePipe para implementar búsqueda por nombre de sede.

3. AreaComponent

- **Ubicación:** src/app/area/area.component.ts

- **Función:**

Similar al componente de Sede, permite gestionar áreas (listar, agregar, eliminar y consultar).

4. CarpetaComponent

- **Ubicación:** src/app/carpeta/carpeta.component.ts



- **Función:**

Permite gestionar carpetas asociadas a áreas o sedes.

Presenta una tabla editable con opciones CRUD.

5. FormatoComponent

- **Ubicación:** src/app/formato/formato.component.ts

- **Función:**

Gestión de documentos o formatos digitales.

Permite visualizar y registrar metadatos como nombre, tipo, y ruta.

6. FilterSedePipe

- **Ubicación:** src/app/pipes/filter-sede.pipe.ts

- **Función:**

Pipe personalizado utilizado para filtrar resultados de sedes según el texto ingresado por el usuario.



➤ Descripción del Componente PanelNavegacionComponent

Este componente actúa como el menú principal de navegación del sistema. Permite al usuario acceder rápidamente a los distintos módulos disponibles: Sede, Área, Carpeta y Formato. Es un componente visual central en la interfaz de usuario.

Archivos que Contiene

- panel-navegacion.component.ts
- panel-navegacion.component.html
- panel-navegacion.component.css
- panel-navegacion.component.spec.ts

Estructura y comportamiento:

- **Visualización:**

Muestra un título centrado "Panel de navegación" y una caja de botones centrada vertical y horizontalmente en la pantalla.

- **Botones de navegación:**

Incluye cuatro botones principales, cada uno con un enlace de enrutamiento (routerLink) hacia los componentes Sede, Área, Carpeta y Formato.

- **Estilos visuales (CSS):**

- Se centra visualmente el contenido con flexbox.
- Los botones están estilizados con colores personalizados y se utiliza Bootstrap para lograr un diseño responsivo y atractivo.



- Se implementan efectos al pasar el cursor (hover) para mejorar la experiencia del usuario.

- **Integración con Bootstrap:**

Se utilizan clases como btn, btn-primary, btn-success, btn-warning, btn-info, btn-block y mb-3 para la disposición y estilización de los botones.

- **Ciclo de vida Angular:**

Implementa el método ngOnInit(), aunque no contiene lógica en este momento. Está disponible para futuras funcionalidades de inicialización si se requieren.

- **Pruebas (spec.ts):**

Contiene una prueba unitaria básica que verifica la correcta creación del componente (should create), asegurando que se renderiza correctamente en la aplicación.

➤ Descripción del Componente SedeComponent

permite la gestión de sedes dentro del sistema. Su interfaz incluye funcionalidades para listar, agregar y buscar sedes por nombre. Utiliza servicios HTTP para comunicarse con la API backend y Bootstrap para el diseño responsivo.

Archivos que Contiene

- sede.component.ts
- sede.component.html
- sede.component.css



- `sede.component.spec.ts`

Estructura y comportamiento

Visualización de sedes:

Al iniciar el componente, se consulta y muestra la lista completa de sedes disponibles mediante una llamada HTTP al backend.

1. **Búsqueda por nombre:**

Permite buscar sedes específicas ingresando un nombre parcial o completo. Si no se encuentra ninguna coincidencia, se muestra una alerta al usuario.

2. **Agregar nueva sede:**

Contiene un formulario simple con validaciones para agregar una nueva sede al sistema. Los datos se envían por HTTP POST al servidor.

3. **Restablecer búsqueda:**

Un botón permite volver a mostrar todas las sedes después de una búsqueda.

Interfaz de usuario (HTML):

- **Secciones claramente diferenciadas:**
 - Botón para regresar al panel principal.
 - Formulario de búsqueda por nombre.
 - Formulario de ingreso de nueva sede.
 - Tabla para mostrar resultados.
- **Diseño adaptado con clases de Bootstrap:**



- Se utiliza btn, form-control, table, thead-dark, entre otras clases para estilizar los elementos.
- La estructura de la tabla está preparada para incorporar futuras funcionalidades como editar o eliminar, aunque actualmente estas no están implementadas.

Estilos (CSS):

- Colores personalizados y estilos consistentes con el tema general de la aplicación.
- Botones con colores distintos según su función (buscar, agregar, regresar).
- Inputs adaptados en tamaño y legibilidad para mejor experiencia de usuario.

Ciclo de vida Angular:

- En ngOnInit() se llama a la función obtenerSedes() para cargar los datos iniciales desde el servidor.

Pruebas (spec.ts):

- Incluye una prueba básica para verificar que el componente se crea correctamente en el entorno de pruebas (should create).



➤ Descripción del Archivo de rutas (AppRoutingModule)

Es el encargado de definir y gestionar la navegación entre los diferentes componentes de la aplicación. Utiliza el sistema de rutas de Angular para redirigir las URLs a los componentes correspondientes, permitiendo una navegación fluida tipo SPA (Single Page Application).

Importaciones clave:

- **NgModule de @angular/core:** Permite declarar el módulo.
- **Routes y RouterModule de @angular/router:** Proveen las herramientas para configurar y exportar las rutas.
- **Componentes de la aplicación:**
 - PanelNavegacionComponent
 - SedeComponent
 - AreaComponent
 - CarpetaComponent
 - FormatoComponent

Definición de rutas (routes: Routes):

1. Ruta raíz (")

- Componente: PanelNavegacionComponent
- Se carga cuando el usuario accede al dominio sin una ruta específica.



- Actúa como página principal o panel de navegación.

2. Ruta /sede

- Componente: SedeComponent
- Carga la interfaz para gestionar las sedes.

3. Ruta /area

- Componente: AreaComponent
- Carga la interfaz para gestionar áreas.

4. Ruta /carpeta

- Componente: CarpetaComponent
- Carga la interfaz para gestionar carpetas.

5. Ruta /formato

- Componente: FormatoComponent
- Carga la interfaz para gestionar formatos.

6. Ruta comodín ('**')

- Redirige cualquier ruta no definida hacia la ruta raíz ("/).
- Funciona como manejo básico de errores de navegación.

Configuración del módulo (@NgModule)



- **imports:**

RouterModule.forRoot(routes)

- Carga la configuración de rutas principales al nivel raíz de la aplicación.

- **exports:**

RouterModule

- Exporta el módulo de rutas para que esté disponible en toda la aplicación, especialmente en AppModule.

COMO INICIAR EL PROYECTO

➤ Pre-requisitos

- Tener instalado XAMPP.
- Colocar la carpeta de la API dentro de la siguiente ruta: \xampp\htdocs.
- Colocar la base de datos en phpMyAdmin utilizando XAMPP.

➤ Iniciar el Back-end

Iniciar Apache y MySQL desde el panel de control de XAMPP haciendo clic en el botón Start.

➤ Iniciar el Front-end

En la terminal de Visual Studio Code (Ctrl + J), ejecutar el siguiente comando: **ng serve**